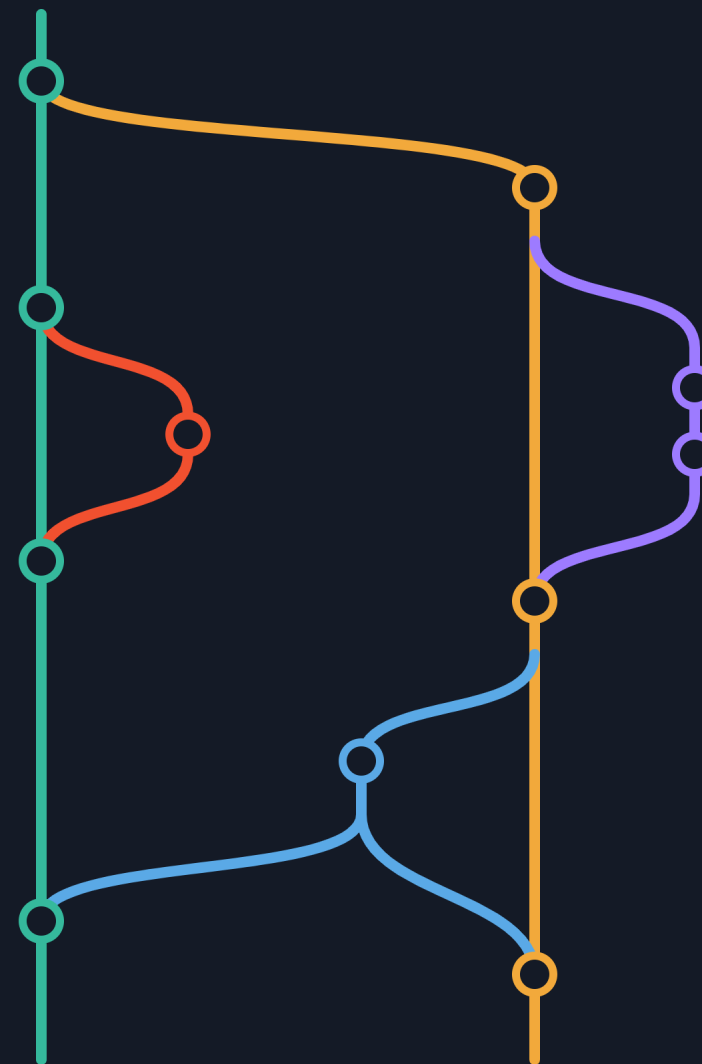


```
$ git switch minicurso
```

Chega de Bagunça no Git

Introdução ao Gitflow

Davi Gabriel Scottini Adriano
UDESC Ibirama · 06 out 2026



QUEM FALA

Davi Gabriel Scottini Adriano

Desenvolvedor PHP

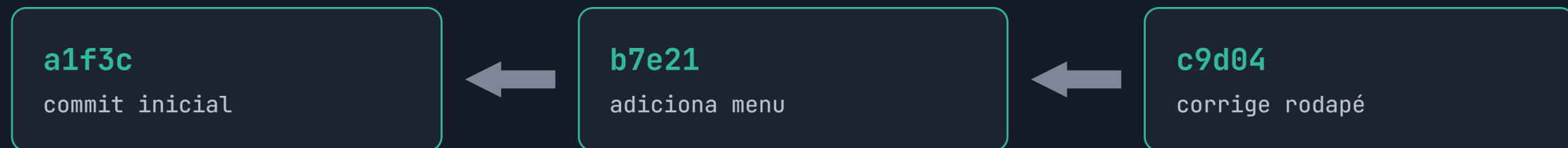
Chega de Bagunça no Git

SUMÁRIO

O que vamos ver

- 01 Revisão rápida de Git
- 02 Branches e ramificações
- 03 Gitflow
- 04 Prática
- 05 Encerramento e perguntas

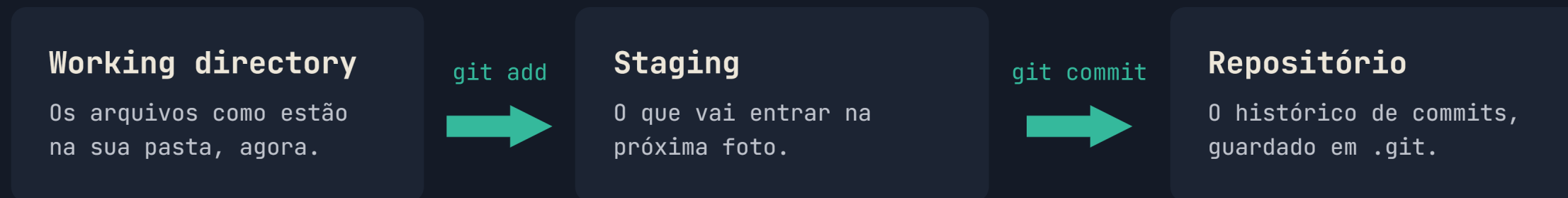
Um commit é uma foto do projeto



Cada commit tem um **hash** e aponta para o seu **pai**. Juntos, eles formam um grafo: o histórico.

```
$ git log --oneline --graph
```

O caminho de uma alteração



`git status` mostra em qual dessas áreas cada arquivo está.

02 • Branches a fundo

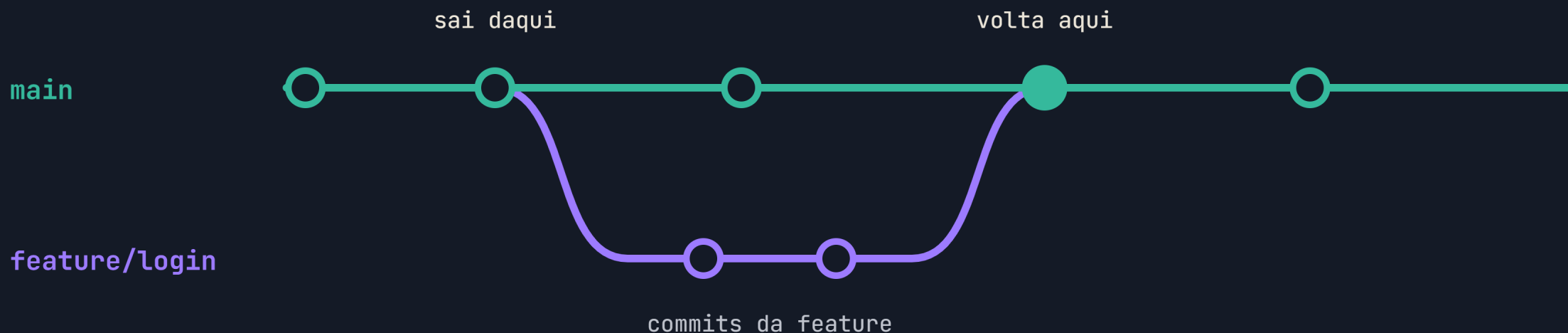
Uma branch é uma *ramificação.*

Um caminho paralelo que sai da linha principal, evolui sem atrapalhar ninguém e depois volta para ela.

BRANCHES

Uma ramificação da main

A branch sai de um ponto da main, recebe seus commits e volta com merge.



```
$ git switch -c feature/login # cria a ramificação
$ git merge feature/login # traz de volta (estando na main)
```

BRANCHES A FUNDO

Linhas paralelas de trabalho

Duas pessoas, duas features, nenhuma atrapalha a outra.



Cada linha evolui sozinha. O **merge** traz a história de volta para a main.

Chega de Bagunça no Git

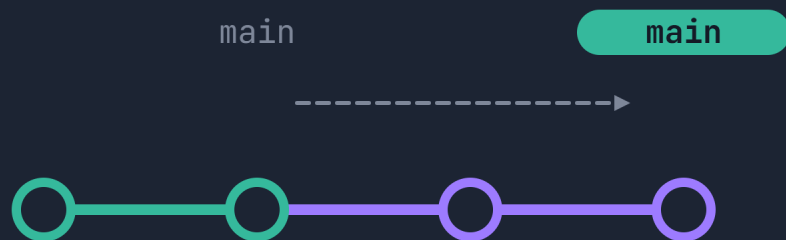
Os comandos do dia a dia

Comando	O que faz
<code>git branch</code>	lista as branches locais
<code>git switch -c nome</code>	cria a branch e já entra nela
<code>git switch nome</code>	troca de branch
<code>git merge nome</code>	traz "nome" para a branch atual

`git checkout` ainda funciona, mas `switch` é mais claro: só serve para trocar de branch.

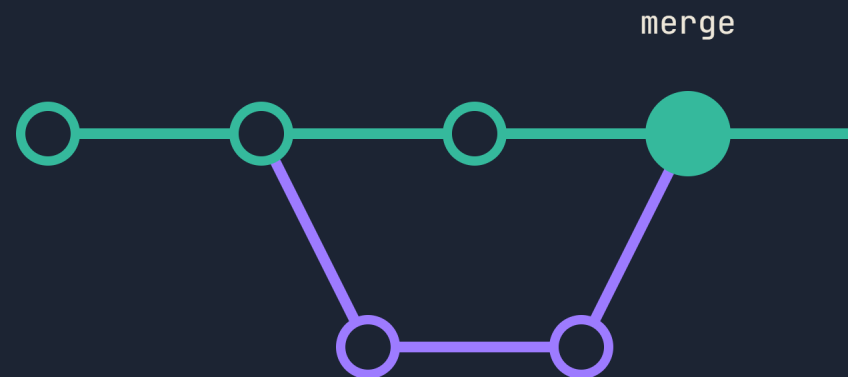
Dois jeitos de fazer merge

Fast-forward



A main não andou desde que a feature saiu. O Git só avança o ponteiro: nenhum commit novo.

Merge de três vias



As duas andaram. O Git cria um commit novo com dois pais.

Conflito: as duas mexeram na mesma linha

```
<<<<<< HEAD
titulo = "Bem-vindo"
=====
titulo = "Olá, turma"
>>>>>> feature/saudacao
```

Como resolver

1. Abra o arquivo marcado como conflito
2. Escolha uma versão, a outra ou junte as duas
3. Apague os marcadores <<< ≡ >>>
4. git add arquivo e git commit

Conflito não é erro. É o Git pedindo para um humano decidir.

BRANCHES A FUNDO

Branches locais e remotas

Seu computador

`main`

`feature/login`

`origin/main` (a última foto do remoto)

`git push →`

`← git fetch`

`← git pull`

`pull = fetch + merge`

GitHub (origin)

`main`

`feature/login`

só existe aqui depois do push

```
$ git push -u origin feature/login # -u liga a local à remota
```

Chega de Bagunça no Git

Provoque um conflito de propósito

1. Crie um repo com saudacao.txt contendo "Olá"
2. Crie a branch ingles e troque para "Hello"
3. Volte para a main e troque para "Oi"
4. Faça o merge de ingles na main
5. Resolva o conflito e finalize com commit

Travou? `git merge --abort` desfaz tudo.

```
$ git init -b main conflito
$ cd conflito
# crie saudacao.txt com "Olá"
$ git add .
$ git commit -m "saudacao"
$ git switch -c ingles
# troque para "Hello" e commite
$ git switch main
# troque para "Oi" e commite
$ git merge ingles
```

```
$ git stash
```

Intervalo

```
Voltamos em 15 minutos.
```

03 • Gitflow

Branches sem regra viram *bagunça*.

Gitflow é um combinado de time: quais branches existem, de onde cada uma sai e para onde ela volta.

GITFLOW

Duas fixas, três de apoio

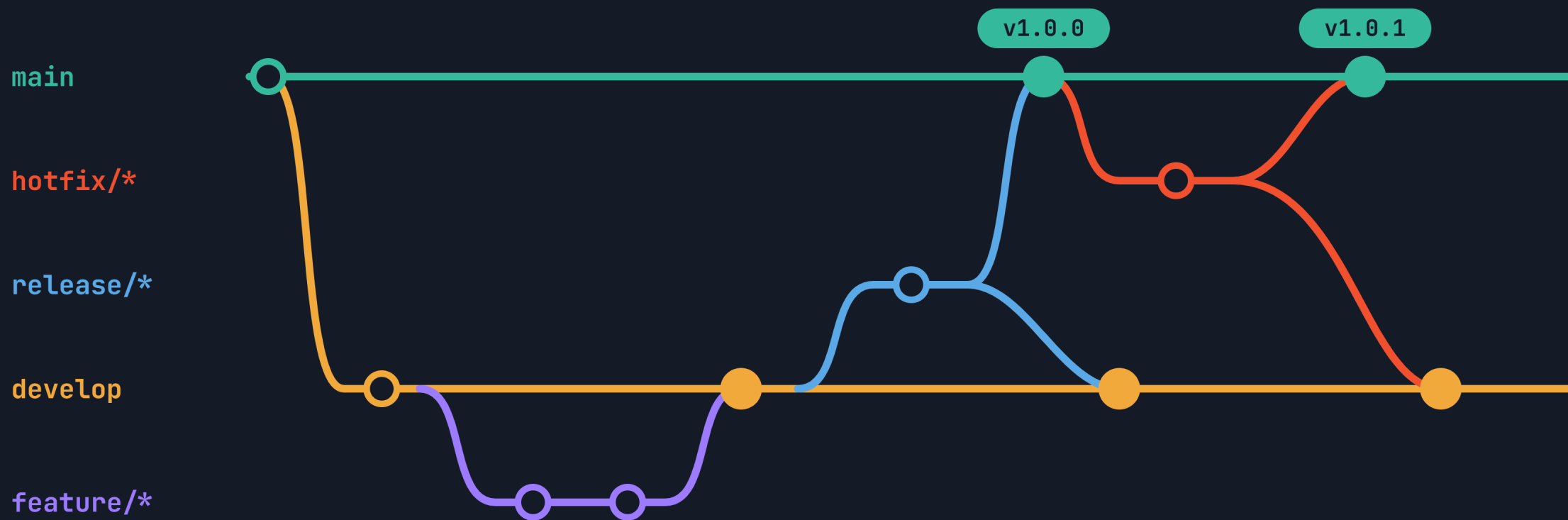
Branch	Sai de	Volta para	Para quê
main	—	—	Código em produção. Cada merge é uma versão.
develop	main	—	Integração do que vai na próxima versão.
feature/*	develop	develop	Uma funcionalidade nova.
release/*	develop	main + develop	Preparar a versão: ajustes finais e número.
hotfix/*	main	main + develop	Correção urgente em produção.

Ninguém commita direto na main nem na develop: tudo chega por merge.

Chega de Bagunça no Git

GITFLOW

O fluxo completo



feature → develop · release → main + develop · hotfix → main + develop

Nem todo projeto precisa de Gitflow

Estratégia	Como funciona	Combina com
Gitflow	main + develop, com release e hotfix	versões numeradas, deploy planejado
GitHub Flow	só main + branches curtas com PR	web apps com deploy contínuo
Trunk-based	todo mundo integra na main várias vezes por dia	times maduros, testes automatizados e feature flags

Saber Gitflow ajuda a entender os outros dois: são versões mais enxutas da mesma ideia.

Cada um no seu repositório

Etapa 1

Estrutura

main + develop

Etapa 2

Features

login e home

Etapa 3

Release

versão 1.0.0

Etapa 4

Hotfix

versão 1.0.1

Antes de começar: **git --version** funciona? Configure seu nome e e-mail:

```
$ git config --global user.name "Seu Nome"
$ git config --global user.email "voce@email.com"
```

Monte a estrutura

Tarefas

1. Crie a pasta gitflow-pratica e inicie o repositório na main
2. Crie um README.md e faça o commit inicial
3. Crie a branch develop a partir da main

Confira: `git branch` mostra main e develop, com * na develop.

```
$ mkdir gitflow-pratica
$ cd gitflow-pratica
$ git init -b main
# crie README.md com "# Minha loja"
$ git add README.md
$ git commit -m "chore: commit inicial"
$ git switch -c develop
```

Duas features

Tarefas

1. Crie feature/login a partir da develop
2. Adicione um login.html e faça pelo menos um commit
3. Volte para a develop e faça o merge com --no-ff
4. Apague a branch da feature
5. Repita tudo com feature/home e um home.html

Confira: `git log --oneline --graph --all` mostra dois merges na develop.

```
$ git switch develop
$ git switch -c feature/login
# crie login.html
$ git add .
$ git commit -m "feat: tela de login"
$ git switch develop
$ git merge --no-ff feature/login
$ git branch -d feature/login
```

Lance a versão 1.0.0

Tarefas

1. Crie release/1.0.0 a partir da develop
2. Crie um arquivo VERSION com 1.0.0 e commite
3. Faça o merge na main e crie a tag v1.0.0
4. Faça o merge também na develop
5. Apague a branch da release

Confira: `git tag lista v1.0.0.`

```
$ git switch -c release/1.0.0 develop
# crie VERSION com 1.0.0 e commite
$ git switch main
$ git merge --no-ff release/1.0.0
$ git tag v1.0.0
$ git switch develop
$ git merge --no-ff release/1.0.0
$ git branch -d release/1.0.0
```

Bug em produção!

Tarefas

1. Crie hotfix/1.0.1 a partir da main
2. Corrija algo no login.html e mude o VERSION para 1.0.1
3. Faça o merge na main e crie a tag v1.0.1
4. Faça o merge também na develop
5. Apague a branch do hotfix

Confira: `git log --oneline --graph --all`. O grafo final é o do slide do fluxo completo.

```
$ git switch -c hotfix/1.0.1 main
# corrija o bug e commite
$ git switch main
$ git merge --no-ff hotfix/1.0.1
$ git tag v1.0.1
$ git switch develop
$ git merge --no-ff hotfix/1.0.1
$ git branch -d hotfix/1.0.1
```

PARA LEVAR PARA CASA

Três ideias

01

Branch é um ponteiro barato. Crie à vontade.

02

Merge junta histórias. Conflito é só uma decisão pendente.

03

Gitflow é um combinado: de onde cada branch sai e para onde volta.

```
$ git commit -m "obrigado!"
```

Perguntas?

Davi Gabriel Scottini Adriano

linkedin.com/in/davi-gabriel-scottini-adriano-329935262

